DATA STRUCTURE COURSE DESIGN PROJECT REPORT

Instructor Name: Professor Sajjad

Semester: 3rd Semester, Second Year

Academic Year: 2025/2026

Project Name:

Mystic Pathways: Your pathway for wider horizons.

Members:

Name	ID	Contribution	Signature
Al-Aswadi Abdulhakim	2024080160	33.3%	Hakim
Al-Sayyadi Ahmed	2024080158	33.3%	Ahmed
Kasasa Musa	2024080159	33.3%	Musa

Project Submission Date: Unknown

Project Duration: Unknown

Contents

Abstract:

We are working on creating an online shop that includes a mini search engine to help have an easier access to the products and items needed by costumers. What makes our project different is that it will have a captivating and efficient interface that highly satisfies the costumer's need.

Introduction

Problem:

The main problem is that in our today's word we lack of effective and dynamic online shops that offer so many categories that satisfy all our needs as human beings in our modern lifestyle.

Project Goal :

The goal of this project is to build a mini online shop in Java with a simple search engine for better practice of mini search engine that uses data structures and sorting algorithms to improve the efficiency and proficiency of product searching in online shops.

Project Objectives :

1. Develop a product catalog based on Java language by using ArrayLists and HashMaps that will store around at least 20 sample products.

2. Implement and apply a search engine that allows users to search products by name and category, achieving at least 80% accuracy in matching results from the product database.(This might be challenging.)

3. Integrate sorting algorithms (by price and name) into the search results to demonstrate the practical application of data structures, so it is not only a university's project, but it is effective for better real-life improvement in the future. Besides, algorithms in improving e-commerce functionality will be efficiently applied.

4. Test the program with at least 10 different user queries to validate search and sorting efficiency, and ensure all coding and documentation are completed by the project deadline with a high quality of efficiency.

Specific Project scope:

1. Data Structures:

Implement a Product class to store product details such as ID, name, category, and price.(Not sure about ID.) The class can be used to store at least 20 products with the specified description.

Use an ArrayList to maintain the product catalog and a HashMap to enable faster category-based searches.

By searching by category returns all matching products correctly, or with a high quality of correction.

2. Search & Retrieval:

Implement a linear search function to find products by full or partial name(Not sure about partial name).

The function retrieves correct matches for at least 10 test queries.

Extend functionality with a binary search for product names after sorting.(Challenging).The binary search correctly finds or rejects products in all test cases.

3. Sorting Algorithms:

Implement sorting of products by price.

Products are displayed in correct order for all test cases.

Implement sorting of products by name (alphabetical ordering).

The output list is correctly sorted in all test cases.

4. User Interface & Output:

Create a menu-driven console interface where users can:

1. View all products

2. Search by name

3. Search by category

4. Sort by price

5. Sort by name

The program runs smoothly, accepts input, and does not crash on invalid entries.Besided, we will work on making it captivating to catch users eyes.

System Design

Data Structure Selection & Justification:

Data Structure	Purpose	Justification	Time Complexity
ArrayList	To store and manage the list of products dynamically.	ArrayList allows dynamic resizing and fast access by index, making it ideal for maintaining product lists that may increase or decrease in size during operation.	

HashMap	To store product details using unique product IDs as keys.	ArrayList allows dynamic resizing and fast access by index, making it ideal for maintaining product lists that may increase or decrease in size during operation.	
Selection Sort Algorithm	To sort products by name or price.	Selection sort is simple to implement and effective for small datasets, making it suitable for the scale of this mini project.	
Linear and Binary Search Algorithms	To locate products by name or category.	Linear search works well for unsorted lists, while binary search significantly improves efficiency when the list is sorted.	

System Architecture:

1. Product Management Module:

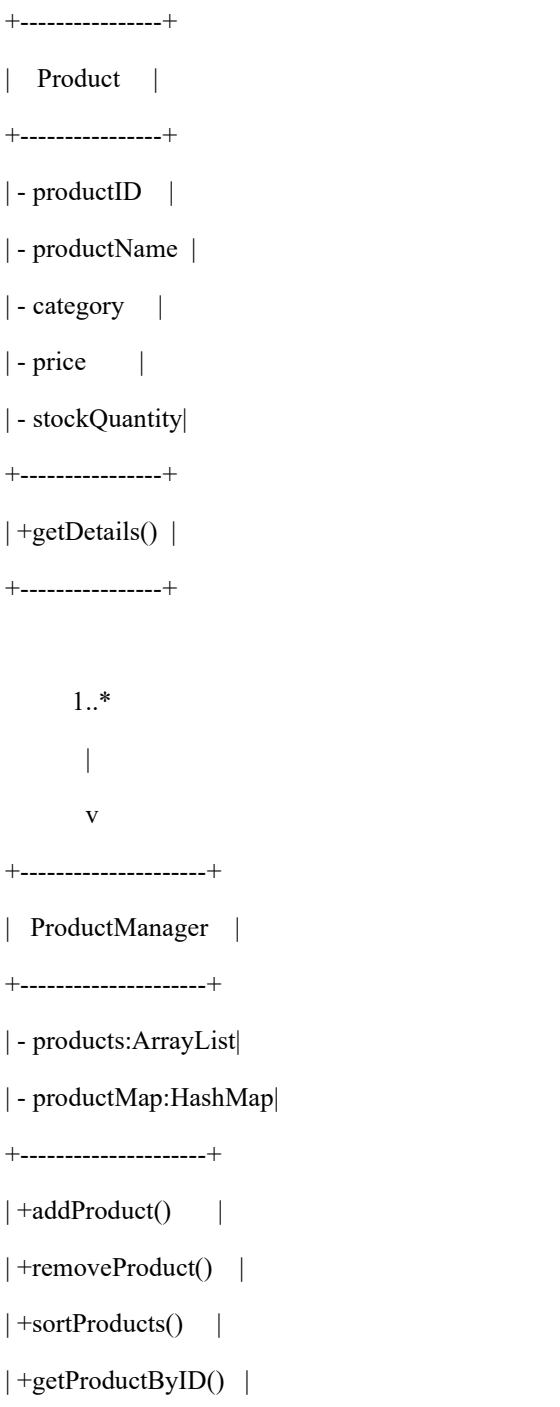
   This component is responsible for handling all product-related operations such as adding, updating, deleting, and displaying product details. It interacts with the data structures (ArrayList and HashMap) to ensure efficient product management and data consistency.

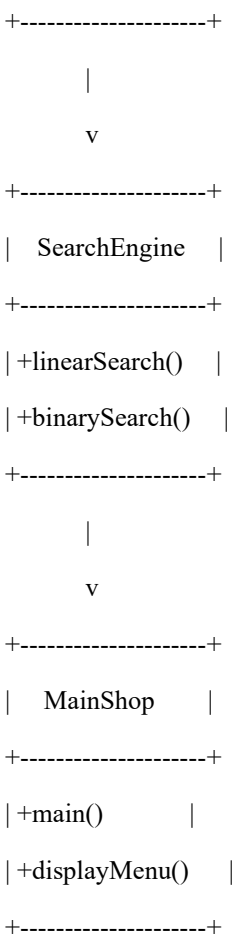
2. Search Engine Module:
   This component enables users to search for products by different attributes such as name, category, or price. It implements linear and binary search algorithms to deliver accurate and fast search results, depending on whether the dataset is sorted.

3. User Interface Module:

This component provides a simple, user-friendly interface that allows users to browse, search, and view product details. It can be implemented using JavaFX or Swing to create an interactive experience that connects the user to the underlying logic and database.

Class Diagram/ System Overview:





Overall System Relationship Description:

The **Maintop** class interacts with both the **Product Manager** and **Search Engine** classes to perform operations requested by the user. The **Product Manager** serves as the central hub for data storage and manipulation, while the **Search Engine** performs the required search operations on the product data. The **Product** class acts as the data model, holding all product-related information. Together, these classes establish a cohesive and efficient structure for managing and searching products in the online shop.

Implementaion:

Key Algorithms:

Algorithm 1: Selection Sort Algorithm

Procedure:

```
procedure selectionSort(productList)
    n ← length(productList)
    for i ← 0 to n - 1 do
        minIndex ← i
        for j ← i + 1 to n - 1 do
            if productList[j].price < productList[minIndex].price then
                minIndex ← j
            end if
        end for
        swap productList[i] and productList[minIndex]
    end for
end procedure
```

Description:
This algorithm sorts the products in ascending order based on price (or name). It repeatedly selects the smallest element from the unsorted portion and places it at the beginning. This sorting helps users view products in an organized manner.

Time Complexity:
$O(n^2)$

Algorithm 2: Linear Search Algorithm

Procedure:

```
procedure linearSearch(productList, keyword)
    results ← empty list
    for each product in productList do
        if keyword is in product.name or product.category then
            add product to results
        end if
    end for
    return results
end procedure
```

Description:
This algorithm searches the product list sequentially to find items matching the user's keyword. It is simple yet effective for small datasets, making it suitable for a mini online shop system.

Time Complexity:
O(n)

Core Functions:

Function	Parameters	Return Type	Description	Time Complexity
addProduct()	String name, double price, String category	void	Adds a new product to the product list.	O(1)
searchProduct()	String keyword	List<Product>	Searches products by name or category using linear search.	O(n)
sortProductsByPrice()	-	void	Sorts all products in ascending order of price using selection sort.	$O(n^2)$
displayProducts()	-	void	Displays all products in a formatted list.	O(n)
getProductById()	int id	product	Returns product details based on its ID.	O(1)